

我们小组从雷霆战机&东方 Project 获取灵感，利用 Qt 环境开发了一款简易的飞机大战游戏。

项目主要包含敌机、自机和子弹三个对象类，并在一个主场景中维护所有的对象。

游戏机制

本游戏延续并结合了雷霆战机和东方 Project 的经典玩法，玩家将操控自机与一众强度逆天的敌机进行对战并角逐出最终胜者。自机通过 WASD 进行移动躲避敌机的子弹，并不断发射子弹来击毁敌机。游戏以波次作为敌机出场的划分，并且以 boss 作为最后一波的标志，玩家需要击毁强大的 boss 来取得游戏的最终胜利。

敌机&&自机

我们发现，敌机和自机两个类具有较为相似的基本性质和行动逻辑，所以我们定义了一个飞机基类，并让敌机类和自机类都继承这一基类。同时我们考虑到，每一个敌机/自机都可以自己发射子弹，所以我们在基类中声明了一个存储每个对象发射出的子弹的链表，链表元素为指向子弹类对象的指针。代码实现如下图。

```
class base_plane_class : public QGraphicsPixmapItem
{
public:
    QList<Bullet*> bullet_list; // 每个机体发射的子弹的链表 需要从playwindow获取
protected:
    int _life; // 血量
    double _speed; // 移动速度
    // 自机可能还需要添加是否处于无敌状态的变量
    bool _camp; // 机体的阵营 自机->SELF 敌机->ENEMY
    bool _state; // 是否活着
public:
    QPixmap pix; // 贴图
    base_plane_class(const QPixmap &p, int l, int s, bool c, QPointF pos, QSize scale, QGraphicsPixmapItem *parent = nullptr)
        : _life(l), _speed(s), _camp(c), _state(true)
    { ...
    }

    virtual void move(double dx, double dy) // 机体向(dx, dy)方向移动一次
    { ...
    }
    virtual int attack(int type, double dx, double dy) // 机体向(dx, dy)方向发射一颗子弹
    { ...
    }
    void bullet_move() { // 子弹移动...
    }
    void bullet_dead() { // 子弹删除...
    }
    virtual void be_attacked() // 机体受击
    { ...
    }
}
```

此外，由于自机是玩家亲自操作而敌机的行动逻辑由我们设计并由程序控制，所以我们将机体的移动、攻击和受击的运行逻辑函数在敌机和自机的派生类中重载，以实现敌机和自机的行动。

对自机而言，机体受击判定是由自机的判定点决定的，所以自机的移动不仅仅需要改变自机贴图位置，还要同步改变判定点的位置。

```
void Myplane::move(double dx, double dy){
    double x0 = this->x();
    double y0 = this->y();
    double nowX = x0 + this->speed() * dx;
    double nowY = y0 + this->speed() * dy;
    if(nowX>=630) nowX=630;
    if(nowX<=-30) nowX=-30;
    if(nowY>=815) nowY=815;
    if(nowY<=-25) nowY=-25;
    this->setPos(nowX, nowY);
    hitPoint->setPos(nowX+32.5,nowY+27.5);
};
```

对于自机的移动速度，我们设计了低速移动模式。在按住'P'键时自机的移动速度为 3 像素/帧，而不按'P'键时自机的移动速度为 8 像素/帧。自机的移动方向根据玩家在'W' 'A' 'S' 'D'四个按键的按键方式决定。代码附下。

```
//自机移动
myplane->setSpeed((myplane->low_speed == true) ? 3 : 8);
for(int i = 0; i < 4; ++i)
{
    double dx = 0, dy = 0;
    dx += (moving[i] ? di[i][0] : 0);
    dy += (moving[i] ? di[i][1] : 0);
    if(dx || dy)
    {
        double len = sqrt(dx * dx + dy * dy);
        myplane->move(dx / len, dy / len);
    }
}
```

对于自机的攻击方式，我们设置为自动攻击，同时在一个阶段的敌机生成结束后升级自机的火力。而自机在受击后，会有五秒钟的无敌时间，在游戏内表现为自机贴图闪烁并在受击后不扣除生命值。

对于敌机，我们总体上设计了三种。一种是不会发射子弹的敌机，通过本身的移动阻挡自机的行动；另一种则是会发射子弹的敌机，还有一种敌机则是 boss。我们一共设计了三种会发射子弹的普通敌机。



这种敌机会向多个角度发射最基础的子弹 🎯



这种敌机在上一种敌机的基础上，还会发射目标为自机判定点的加速自机狙 🔫



这种敌机会想多个角度发射可以在版边反弹一次的子弹 🏀，而在敌机死亡后会产生亡语，向周围发射子弹 🌪️

```

if(type==EMYBULLET_FIRST){
for (int i = 0; i < 6; ++i) {
    Bullet *b = new Bullet(type,-sin(qDegreesToRadians(45-18*i)),cos(qDegreesToRadians(45-18*i)));
    b->setPos(this->x() + this->pix.width() / 2 - b->size[0] / 2,this->y() + this->pix.height());
    bullet_list.append(b);
}
return 6;
} else if(type==EMYBULLET_SECOND){
    for (int i = 0; i < 4; ++i) {
        Bullet *b = new Bullet(type,-sin(qDegreesToRadians(30-20*i)),cos(qDegreesToRadians(30-20*i)));
        b->setPos(this->x() + this->pix.width() / 2 - b->size[0] / 2,this->y() + this->pix.height());
        bullet_list.append(b);
    }
return 4;
} else if(type==EMYBULLET_THIRD){
    Bullet *b = new Bullet(type,dx,dy);
    b->setPos(this->x() + this->pix.width() / 2 - b->size[0] / 2,this->y() + this->pix.height());
    bullet_list.append(b);
return 1;
}

```

关于 boss 设计，我们还添加了两种子弹，一种是会不断加速的连射自机狙；

另一种则是太阳系模型类子弹，如图所示。



子弹

子弹的种类及运行逻辑在上文中已经介绍过了，这里主要介绍子弹类对象的实现。我们根据子弹在游戏中所需要的性质定义了子弹类

```
class Bullet : public QGraphicsPixmapItem
{
public:
    Bullet(const int ty,const double dx,const double dy,QGraphicsPixmapItem *parent = nullptr);
    Bullet(const Bullet &bul, QGraphicsPixmapItem *parent = nullptr);
    int type;//子弹类型, 据此写子弹pixmap及其大小 switch type: //TODO
    double bullet_speed;//弹速
    bool state;//子弹是否仍然存活
    double dir[2];
    bool camp;//子弹的阵营, 敌0自1
    int size[2];
    double a = 0;//子弹加速度
    int bounce_times =1;
    void move(const int screen_x,const int screen_y);
    Bullet* orb[3] = {nullptr};
    double angle;

signals:
};
```

不同种子弹的主要区别就在于运动逻辑。代码实现如下。

```
if(type==SLFBULLET||type==SLFBULLETLEVELUP){
    nowX = this->pos().x();
    nowY = this->pos().y()-bullet_speed;
    this->setPos(nowX,nowY);
}
else if(type==BOSSBULLET_SECOND){
    nowX = this->pos().x() + dir[0]*bullet_speed;
    nowY = this->pos().y() + dir[1]*bullet_speed;
    this->setPos(nowX,nowY);
    double centerX = this->pos().x()+this->size[0]/2;
    double centerY = this->pos().y()+this->size[1]/2;
    angle += a;
    a += 0.01;
    for(int i = 0;i<3;i++){
        if(orb[i]){
            nowX = centerX + (30*cos(qDegreesToRadians(angle))+70)*sin(qDegreesToRadians(angle+120*i)) - orb[i]->size[0]/2;
            nowY = centerY + (30*cos(qDegreesToRadians(angle))+70)*cos(qDegreesToRadians(angle+120*i)) - orb[i]->size[1]/2;
            orb[i]->setPos(nowX,nowY);
        }
    }
}
else {
    if(type==EMYBULLET_THIRD||type==BOSSBULLET_FIRST){
        bullet_speed+=a;
    }
    nowX = this->pos().x() + dir[0]*bullet_speed;
    nowY = this->pos().y() + dir[1]*bullet_speed;
    this->setPos(nowX,nowY);
}
if(this->type == BOSSBULLET_SECOND){
    if(nowX<-size[0]-100||nowX>screen_x+100||nowY<-size[1]-100||nowY>screen_y+100){
        state = 0;//出界, 删去子弹
    }
}
if(type==EMYBULLET_SECOND && bounce_times && (nowX<0||nowX>screen_x-size[0])){
    bounce_times--;
    dir[0]*=-1;
}
if(nowY<-size[1]||nowY>screen_y||nowX<-size[0]||nowX>screen_x){
    state = 0;//出界, 删去子弹
}
```

主场景

我们在主场景中定义了一个计时器 timer, 以每一帧的时间为周期更新场景中的各对象, 包括自机、敌机和子弹, 同时在每次经过一定时间后生成新的敌机、改变游戏阶段。

对于敌机&敌机子弹击中自机以及自机子弹击中敌机的判定, 我们调用了 Qt 内置的 QGraphicsItem 类的内置函数'collidesWithItem'进行判断, 并及时移除发生碰撞的子弹以及移出版面的敌机和子弹。

分工

陈晓东——场景绘制、敌机及子弹设计及实现、视频配音

李建霖——策划、图片素材贡献者、自机及子弹设计及实现

赵思棋——关卡设计、敌机生成逻辑搭建

王彦博——策划、敌机设计及实现、实机游玩